

Let's citrusize pangenome graphs.

Shuto Machida ¹, Mayumi Kamada ², Yuki Moriya ³, and Toshiaki Katayama ³

BioHackathon series:

DBCLS BioHackathon 2026

Matsuyama, Japan, 2026

[mikan-pangenome](#)

Submitted: 25 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

¹ Kyushu University ² Kitasato University ³ Database Division for Life Science (DBCLS), BioData Science Initiative (BSI), National Institute of Genetics, Research Organization of Information and Systems, Chiba, Japan

Introduction

Almost all routine genome analysis is still organised around a single reference genome: reads from a new individual are mapped to one assembly, differences from that assembly are called, and the individual is described by the resulting difference set. This works, but it fails in three ways that matter. Sequence that is absent from the reference is effectively invisible — an insertion carried by the sample simply has nowhere to map, and any gene inside it is silently lost. The reference itself is one individual, not an average: GRCh38 matches nobody exactly, and in crops a reference such as Citrus Clementine v1.0 differs from Satsuma mandarin at millions of loci, so that describing one cultivar as a list of “variants” against another is both inefficient and misleading. And population diversity — the thing breeding and population genetics actually need — cannot be represented by any single individual at all.

Pangenomes were introduced to address exactly this (Tettelin et al., 2005). A pangenome is a data structure that represents the genetic material present in a population rather than in one individual. Historically three representations have been used: gene-based (presence/absence of genes, partitioned into core, accessory and private (Golicz et al., 2016)), sequence-based (all assemblies retained, redundancy compressed), and graph-based, in which shared sequence is collapsed into nodes and differences appear as branches. The graph representation is now the mainstream choice, because it expresses SNPs, indels, structural variants and presence/absence variation in one uniform structure, and because it permits reference-free analysis: no single individual has to serve as the backbone. The draft human pangenome reference (Liao et al., 2023) and graph-based plant pangenomes, including one for the orange subfamily (Huang et al., 2023), illustrate what becomes visible once the reference bias is removed.

Graph pangenomes have been conceptually available since 2005, but only recently became practical to build, for three reasons. First, long-read sequencing at scale: PacBio HiFi and Oxford Nanopore ultra-long reads deliver 15–25 kb reads at QV > 40, so repeats assemble correctly and large structural variants are spanned by single reads. Second, haplotype-resolved assembly: HiFi with Hi-C, or trio phasing, now separates the two haplotypes of a diploid into two assemblies, which is a precondition for capturing heterozygous variation honestly. Third, mature construction tools — minigraph (Li et al., 2020), Minigraph-Cactus (Hickey et al., 2024) and PGGB (Garrison et al., 2024) — that run on ordinary HPC hardware in hours to days.

What has not kept pace is the path into the field for newcomers. Building a graph in practice requires knowledge that is scattered across specifications and tool documentation: the PanSN sequence-naming convention, the fact that .gfa denotes at least three mutually incompatible dialects (GFA v1.0 with P lines from PGGB, GFA v1.1 with W lines from Minigraph-Cactus, and reference-centric rGFA with no path lines at all from minigraph), the decision of whether to build per chromosome or over the whole genome, and the parameters that govern alignment identity and segment length. Much of this is learned only by failing, and the failures are

typically silent: a file parses cleanly but contains zero paths; a parameter is accepted but not applied, and the graph is merely under-compressed rather than broken. Existing tutorials tend to use toy data at a scale where such problems do not surface, and to stop at “the command ran”.

We took Japanese mandarins as a case where these gaps can be closed with real data at a tractable scale. Satsuma mandarin (*Citrus unshiu*) is an F1 of Kishu mandarin (*C. kinokuni*) × Kunenbo (*C. nobilis* var. *kunip*), and haplotype-resolved, chromosome-scale assemblies of all three cultivars have been published and made available through Plant GARDEN (Isobe et al., 2023). Three cultivars × two haplotypes gives six haploid genomes of roughly 300–360 Mb over nine chromosomes — large enough to be a genuine pangenome run, small enough for a student to complete on a shared HPC. Crucially, the pedigree supplies ground truth: each Satsuma haplotype should resemble one parent, so the constructed graph can be checked against a biological expectation rather than only against internal quality metrics.

This report describes what we built around that dataset at the DBCLS BioHackathon 2026. Our objectives were threefold. (i) To produce an open, reproducible tutorial that takes a reader from the concept of a pangenome through data acquisition, QC, PGGB graph construction, graph QC and biological interpretation, with all scripts included and the text available in both Japanese and English. (ii) To record the pitfalls explicitly — the version-specific silent parse failure of a PGGB parameter, haplotype leakage visible as duplicated BUSCO genes, GFA dialect mismatches, and the quality thresholds by which such problems can be recognised — so that they become teachable material instead of private lore. (iii) To make the resulting graphs explorable in a browser, by developing the Unshu Haplotype Viewer, which places the six haplotypes in a common coordinate system and exposes shared nodes, non-reference branches, path orientation and local similarity to each parental lineage.

Materials and methods

Assembly data

All analyses use published, haplotype-resolved, chromosome-scale assemblies of Satsuma mandarin (*Citrus unshiu*, CUN) and its two parents, Kishu mandarin (*C. kinokuni*, CKI) and Kunenbo (*C. nobilis* var. *kunip*, CKU) (Isobe et al., 2023). Two releases were used. Release r1.0 was obtained from Plant GARDEN (Kazusa DNA Research Institute) on 14 September 2026; release r2.0 was obtained from the Mikan Genome Database 2 (MiGD2, NARO) on 16 September 2026, the release itself being dated 23 March 2026. The six haploid assemblies of each release are listed in Table 1.

Table 1: Assemblies used in this project. Sizes are restricted to chromosomes 1–9, which is what enters the graph. “Unplaced in file” gives the unplaced sequence and contig count present in the downloaded file; in r1.0 unplaced contigs are distributed separately and were not part of the graph input, so their absence here is a property of our download, not of the source. The full record — distributor, landing page, dataset ID, distributed filename, download date, byte count, SHA-256, record count and provenance of every field — is in `tables/assembly_provenance_v1_v2.tsv` in the tutorial repository.

Release	PanSN	Assembly	Cultivar	Dataset ID	chr1–9 (Mb)	N50 (Mb)	Unplaced in file (Mb / contigs)
r1.0	CUN#1	CUN-phKu	Satsuma	t55188.G004	357.6	44.2	n/a
r1.0	CUN#2	CUN-phKi	Satsuma	t55188.G003	348.5	34.5	n/a
r1.0	CKI#1	CKI-hap1	Kishu	t408488.G002	304.2	33.5	n/a

Release	PanSN	Assembly	Cultivar	Dataset ID	chr1–9 (Mb)	N50 (Mb)	Unplaced in file (Mb / contigs)
r1.0	CKI#2	CKI-hap2	Kishu	t408488.G003	310.3	32.5	n/a
r1.0	CKU#1	CK-Uhap1	Kunenbo	t481549.G002	323.9	36.6	n/a
r1.0	CKU#2	CK-Uhap2	Kunenbo	t481549.G003	303.4	33.0	n/a
r2.0	CUN#1	CUN-phKu	Satsuma	CUN-phKu_r2.0	300.4	31.5	7.3 / 78
r2.0	CUN#2	CUN-phKi	Satsuma	CUN-phKi_r2.0	298.7	31.1	20.4 / 265
r2.0	CKI#1	CKI-hap1	Kishu	CKI-hap1_r2.0	295.6	30.5	13.0 / 142
r2.0	CKI#2	CKI-hap2	Kishu	CKI-hap2_r2.0	297.9	30.1	17.5 / 186
r2.0	CKU#1	CK-Uhap1	Kunenbo	CK-Uhap1_r2.0	295.1	30.8	21.9 / 203
r2.0	CKU#2	CK-Uhap2	Kunenbo	CK-Uhap2_r2.0	306.4	33.5	16.0 / 82

Haplotype numbering in PanSN is arbitrary, so we fix it once and keep it throughout: CUN#1 is CUNphKu, the Kunenbo-derived Satsuma haplotype, and CUN#2 is CUNphKi, the Kishu-derived one. The assignment is made in `tables/samplesheet.tsv` and propagates to the graph paths, the VCF coordinate reference and the direction of every pedigree test reported below.

Input preparation and graph construction

Sequences were renamed to PanSN form (sample#haplotype#contig, e.g. CUN#1#chr01), upper-cased — r2.0 is distributed repeat-masked in lower case, r1.0 is not, and the aligner should see both releases on the same terms — and split into one bgzip-compressed FASTA per chromosome containing the six haploid sequences; the preparation step extracts chromosomes 1–9 only and warns if the number of records extracted is not nine.

Pangenome graphs were built per chromosome with PGGB (Garrison et al., 2024), run from a Singularity image of `ghcr.io/pangenome/pggb` (revision 4225c6c), with `-p 95 -s 10000 -n 6 -B 1G -V CUN#1` and 32–48 threads per job. The identity threshold `-p 95` sits a few points below the intra-specific identity of citrus (SNP density of about 17/kbp, i.e. ~98.3% identity); the segment length `-s 10000` is the usual starting point for chromosome-scale assemblies and was checked by a sweep on chromosome 9. `-B` is passed explicitly because the default is not parsed correctly in this build (see Results). The nine chromosomes run as independent jobs and complete in 5–10 hours in total on a shared HPC node.

Graph QC and downstream analysis

Graphs were evaluated on four levels. Structure was read from `odgi stats -S` (Guarracino et al., 2022), from which we compute the compression ratio (total graph bp divided by total input bp) and check that the graph carries exactly six paths. Path preservation was checked by extracting paths with `odgi paths -f` and comparing their lengths with the input FASTA. Pedigree consistency was assessed from pairwise Jaccard similarities between paths (`odgi similarity`), compared in MAX-based form: each Satsuma haplotype is required to be more similar to the closer haplotype of its expected parent than to either haplotype of the other parent. Variants were called from the graph with `vg deconstruct -P CUN#1 -H '#' -a -e`

(Garrison et al., 2018) and classified as SNPs, indels or structural variants (≥ 50 bp) by allele length. In addition, the value of `transclose-batch` actually used was read back from the `*.params.yml` file that PGGB writes, for every chromosome and both releases.

Ancestry along each Satsuma haplotype was called in 100 kb windows of the CUN#1 coordinate system of the release in question, each window being assigned to the parental haplotype with which it shares the most graph sequence, or left as “no call” where the signal does not separate the candidates. The two releases were compared window by window after matching the arbitrary haplotype numbering between them.

For the release comparison, the entire pipeline was pinned: the same tool versions and the same 36 scientific parameters were used for r1.0 and r2.0, verified from the parameter records written by the tools themselves rather than from the command lines issued. The assemblies are therefore the only thing that differs between the two runs.

Results

An end-to-end tutorial on a pedigree-backed citrus dataset

The first outcome of the project is the tutorial itself, released as a public repository (<https://github.com/lid-lab/citrus-pangenome-tutorial>; text under CC BY 4.0, code under the MIT licence). It is written for advanced undergraduates who know what a FASTA file and an assembly are, are comfortable on the Linux command line, and have never built a pangenome. The text exists in parallel Japanese and English versions, and every command shown in the text is also available as a script in `scripts/`, so that a reader can either type along or run the pipeline end to end.

Table 2: Structure of the tutorial. Each chapter is available in Japanese (`docs/ja/`) and English (`docs/en/`).

Chapter	Topic	What the reader does
0	What is a pangenome	Concepts: reference bias, the three representations, why graphs
1	General workflow	Cross-tool vocabulary: PanSN, GFA dialects, node/edge/path, compression ratio
2	Target organisms	Satsuma = Kishu $\times$ Kunenbo, trio phasing, what a haplotype is and is not
3	Data acquisition	BioProject/BioSample, Plant GARDEN, downloading the six haploids
4	Data QC	<code>seqkit stats</code> , expected values per species, anomaly detection
5	Graph construction	Tool comparison, PGGB parameters, running on an HPC
6	Graph QC	Four-level evaluation, compression ratio, parameter records
7	Graph interpretation	VCF, visualisation, pedigree verification, r1.0 vs r2.0

Chapter	Topic	What the reader does
Appendix	Minigraph-Cactus	The same data through MC (described, not executed)

The dataset is the published assembly set of Satsuma and its two parents (Methods, Table 1): three cultivars $\times$ two haplotypes, nine chromosomes each. Because Satsuma was trio-phased against both parents (Cheng et al., 2021), every one of its haplotypes has a known parent of origin, and that is what makes the dataset worth teaching on — the finished graph can be asked to confirm something it was not given.

Graphs are built per chromosome (Methods), which makes nine independent jobs of six sequences each, 54 in total, and completes in 5–10 hours on a shared HPC node. The tutorial then evaluates the result on four levels, which is the framework we propose as the teachable core of graph QC: **structure** (node, edge and path counts; compression ratio), **preservation** (do the six input haplotypes come back out of the graph at their original lengths), **biological validity** (do path similarities respect the known pedigree), and **SV signal** (are variant counts and the size distribution plausible). Failures at the first two levels are fatal — the graph is wrong — whereas failures at the last two are caveats on interpretation, and the tutorial treats them that way.

On the r1.0 data the graphs pass. Compression ratios for the nine chromosomes fall between 0.186 (chr4) and 0.278 (chr8), comfortably inside the 0.19–0.30 band expected for closely related haplotypes; all six input paths are recovered with a maximum length difference of 0.000%; and the pedigree test succeeds in its MAX-based form, with each Satsuma haplotype most similar to a haplotype of the expected parent. Variant calling with `vg deconstruct` (Garrison et al., 2018) against the CUN#1 coordinate system yields, for example, 432,698 records on chr09 — a number that is only interpretable once one knows that multi-allelic sites are expanded, which is itself one of the lessons below.

Reproducibility is handled by construction rather than by exhortation. Every path into the pipeline comes from `tables/samplesheet.tsv`, so switching to a different assembly release means editing one table and nothing else; `tables/assembly_provenance_v1_v2.tsv` records the provenance and the **SHA-256** of all twelve assemblies (Table 1), so that a reader can verify that they are holding the same bytes we were; and the quality checks read the parameter files the tools themselves wrote out, rather than the command line the operator believes they typed.

Pitfalls and practical caveats

The second outcome is a catalogue of the things that actually went wrong, or nearly did, while running this analysis on real data. We list them here because they share a property that makes them hard to learn from documentation: **they do not announce themselves**. The exit code is zero, the error log is empty, and the damage is visible only as a number that is a little off.

Table 3: Practical pitfalls encountered in this project, how each one is detected, and the response.

Pitfall	How it presents	Detection	Response
PGGB <code>-B</code> (transclose-batch) default fails to parse in some builds	Run succeeds; graph is under-compressed	<code>transclose-batch</code> in <code>*.params.yml</code> ; compression ratio > 0.9	Always pass <code>-B</code> explicitly (<code>-B 1G</code>) and verify it in the parameter record

Pitfall	How it presents	Detection	Response
GFA dialect mismatch	Tool reports zero paths on a valid .gfa	<code>grep -c '^P'</code> vs <code>grep -c '^W'</code>	<code>vg convert -g in .gfa -f -W</code> to write P lines; rGFA has no paths at all
Arbitrary PanSN haplotype numbering	Two analyses of the same data disagree in direction	Which file is <code>hap1_path</code> in the samplesheet	Fix and document the mapping (here <code>CUN#1 = CUNphKu</code> , <code>CUN#2 = CUNphKi</code>); check it before comparing with others
Unequal path lengths distort Jaccard	Pedigree test fails although the graph is fine	AVG-based test NO, MAX-based test YES	Use MAX-based comparison; longer paths inflate the union and depress Jaccard
Multi-allelic expansion in <code>vg deconstruct</code> Haplotype leakage / duplication	Variant counts an order of magnitude above expectation One haplotype larger than its partner	Compare with and without <code>-e</code> <code>seqkit stats</code> size symmetry; BUSCO duplicated fraction; Merqury from raw reads	Read counts as records, not as independent variants Flag as a data-side caveat; do not silently attribute it to the graph
Assembly release drift	Sizes and coordinates differ from the publication	Provenance table, SHA-256, distributor's release notes	Check for a newer release before starting; pin what you used
Repeat masking and unplaced contigs	Aligner sees lower-case sequence; extra records enter the graph	Record counts after input preparation	Upper-case the sequence, extract chr1–9 explicitly, warn if the count is not 9

Four of these deserve more than a table row.

Silent parameter failure. In the PGGB build we used ([ghcr.io/pangenome/pggb:latest](https://github.com/pangenome/pggb), revision 4225c6c approx.), the default value of the `-B` (transclose-batch) option is not parsed correctly and seqwish silently falls back to a much smaller value, leaving the graph under-compressed. Nothing in the exit status or the logs says so. This was diagnosed by systematic parameter experiments rather than by reading an error message, and the general lesson we draw in the tutorial is to **verify parameters from the records the tools write, not from the command you meant to run** — the `.params.yml` file is authoritative, memory is not. The cost of finding out late is aggravated by the shape of the pipeline: PGGB performs alignment, graph induction and smoothing in one command, so discovering that a graph-induction parameter was wrong means repeating the wfmask alignment, by far the most expensive stage.

Metrics that are sensitive to things you did not intend to measure. In r1.0 the two Satsuma haplotypes are 25–55 Mb larger than the parental assemblies. Because Jaccard similarity is intersection over union, a longer path enlarges the union and depresses the coefficient, so an average-based pedigree test fails on a graph that is perfectly sound. The same comparison in MAX-based form passes. Similarly, Satsuma haplotypes are recombinant mosaics of the two parental haplotypes, so averaging similarity over both of a parent's haplotypes flattens

exactly the structure one is trying to see. The tutorial therefore teaches the distinction between “which parent” (which a whole-chromosome comparison can establish) and “which parental haplotype” (which it cannot, and which needs a windowed analysis).

An assembly is a versioned artefact. Midway through the project, NARO released r2.0 of all three cultivars on MiGD2 (23 March 2026). Rebuilding the graphs from r2.0 with the same pipeline and the same 36 parameters and tool versions showed, first, that the size anomaly disappears — over chromosomes 1–9 the six haploids span 295.1–306.4 Mb in r2.0 against 303.4–357.6 Mb in r1.0 — and second, that the ancestry mosaic is essentially unchanged: 3268 of 3300 100-kb windows agree (99.0%) across the 16 track $\times$ chromosome pairs whose parental phase is stable between releases, and 3539 of 3733 (94.8%) over all 18 pairs. The two exceptions are chromosomes 1 and 9 of the Kishu-derived Satsuma haplotype, where the *parental* assembly’s own phase is reorganised between releases — the caveat about mistaking a phasing switch for a crossover, actually happening. Against an independent yardstick, a RAD-Seq F1 population of 96 genuine Kunenbo $\times$ Kishu offspring, Satsuma sits inside the distribution of real offspring under both releases, at the same percentile (93.8). We therefore do not claim that r2.0 corrected r1.0: there is no independent ground truth for Satsuma, and which release is closer to biology cannot be decided from this comparison. What the exercise does establish is methodological (Figure 1) — that a quantity as direct as assembly length moved by 16% while a conclusion resting on relative comparison did not move at all, and that the attribution was only possible because exactly one thing was varied.

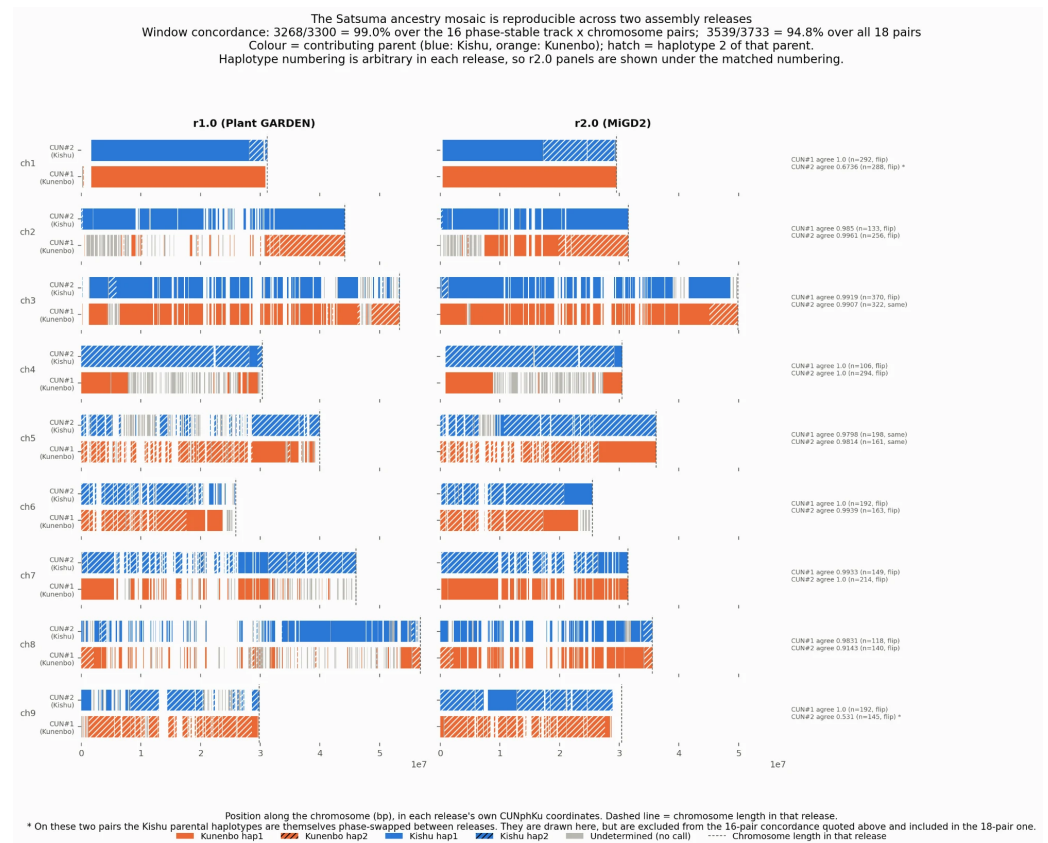


Figure 1: Figure 1. The Satsuma ancestry mosaic called independently from the two assembly releases with an identical pipeline. Each pair of tracks gives the two Satsuma haplotypes along one chromosome, in that release's own CUN#1 coordinates (dashed line = chromosome length in that release). Colour is the contributing parent (blue, Kishu; orange, Kunenbo), hatching marks haplotype 2 of that parent, and grey is a window with no call. Window concordance between the releases is 3268/3300 = 99.0% over the 16 phase-stable track x chromosome pairs and 3539/3733 = 94.8% over all 18; the two excluded pairs (marked * in the figure, CUN#2 on ch1 and ch9) are those where the Kishu parental haplotypes are themselves phase-swapped between releases. Haplotype numbering is arbitrary in each release, so the r2.0 panels are shown under the matched numbering.

Record the oddity; do not explain it away. The size anomaly was logged in the tutorial's QC chapter without a resolution, because the data at hand could not settle it. It was settled later, by a new release, not by our reasoning. Had it been sealed with a plausible story at the time, the chance to reconcile it would have been lost. We consider this the single most transferable habit in the material, and it is the reason the pitfalls above are written into the tutorial as content rather than kept as an errata list.

Pangenome Viewer

To enable browser-based exploration of the chromosome-specific GFA files generated in the preceding analyses, we developed a program that extracts shared nodes, non-reference sequences, path orientations, within-path positions, and local similarity measures and converts them into lightweight JSON files. Using the Satsuma mandarin CUNphKu haplotype (CUN#1) as the reference coordinate system, we developed the Unshu Haplotype Viewer, which allows six haplotypes from Satsuma mandarin, Kishu mandarin, and Kunenbo mandarin to be compared within a common coordinate system (Figure 2).

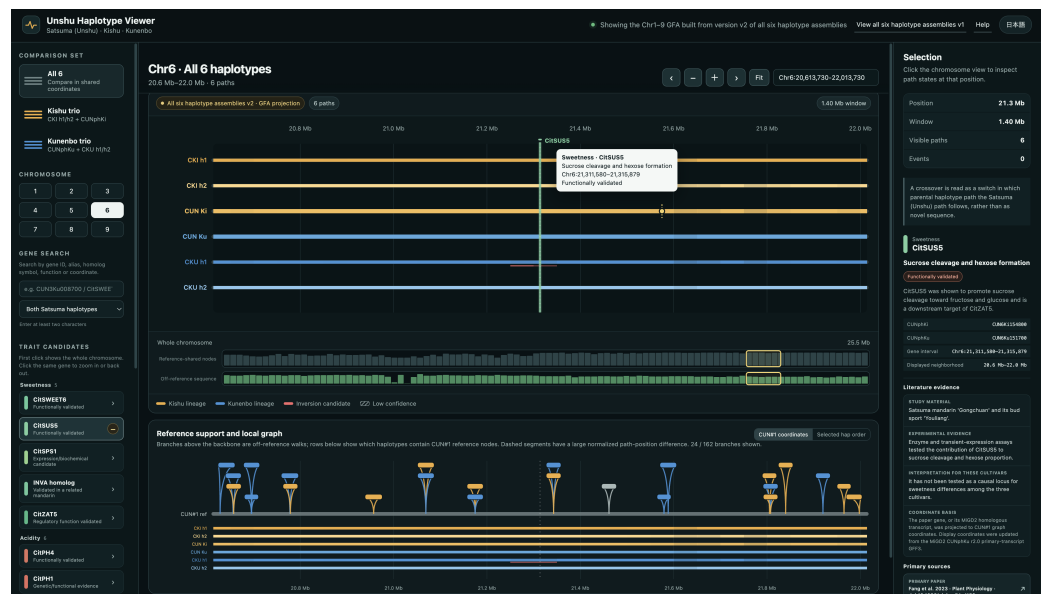


Figure 2: Figure 2. The Unshu Haplotype Viewer. The viewer presents the six haplotype paths, reference-sequence support, non-reference branches, local similarity to the parental lineages, and candidate-gene search results within the same genomic interval.

The viewer provides both a combined display of all six haplotypes and two three-haplotype views that compare each Satsuma haplotype with either the Kishu or Kunenbo lineage. It supports chromosome-wide overviews, interval zooming, horizontal panning, reference-node coverage, candidate inverted regions, the amount of non-reference sequence, and local graph visualization. In the local graph, sequences following the reference path are distinguished from non-reference branches, and each branch is coloured according to the haplotypes that traverse it. Shared-node similarity between each Satsuma haplotype and the four parental haplotypes is also displayed along the chromosomes, allowing users to examine whether a region is more similar to the Kishu or Kunenbo lineage. These views facilitate visual exploration of candidate switches in parental origin, structural variation, and potential assembly or phasing inconsistencies.

To facilitate investigation of genotype–phenotype relationships, we registered literature-derived candidate genes associated with sweetness, acidity, bitterness, aroma, peel colour, fruit size, seedlessness, polyembryony, and bioactive compounds. We also constructed a search index containing 49,322 genes from the MiGD2 CUNphKi and CUNphKu v2 primary-transcript annotations. Users can navigate to any annotated gene by searching for a gene ID, transcript ID, homologous-gene symbol, functional description, or genomic coordinate. CUNphKu annotation coordinates are used directly, whereas CUNphKi coordinates are projected onto CUN#1 using shared GFA nodes. The viewer reports the projection method and shared-node coverage. Genes that cannot be placed on chromosomes 1–9 remain available in the search index.

A Help page describing the calculations and their interpretive limitations was included, together with a separate URL for accessing the previous assembly version. The viewer was published through GitHub Pages, and the source code, display data, and data-conversion programs were released in a public GitHub repository.

GitHub repository: <https://github.com/moriya-dbcls/unshu-haplotype-viewer>

Public viewer: <https://moriya-dbcls.github.io/unshu-haplotype-viewer/>

Shared-node similarity and path switching do not directly establish parental origin. They were therefore treated as exploratory measures for identifying candidate regions that require

further validation through nucleotide-level alignment, structural-variant analysis, and HiFi read support.

Database and visualization-tool development has traditionally emphasized generic designs intended for long-term reuse. In contrast, the principal components of this viewer were implemented within several hours through interaction with AI, demonstrating that interfaces tailored to a particular dataset and research question can now be developed rapidly. This does not mean that general-purpose design is no longer necessary. Standardization, maintainability, and reusability remain essential for infrastructure intended to support multiple studies and datasets over an extended period. For exploratory research with a restricted scope, however, rapidly developing a purpose-specific viewer may be more efficient. Future development should therefore distinguish between components that require a reusable, general-purpose foundation and those that are better implemented for a specific research question, according to the expected lifetime of the resource, its intended users, the diversity of the input data, and the goals of the analysis.

Discussion

The three outcomes reported here — the tutorial, the catalogue of pitfalls, and the viewer — were produced from one analysis, and they are more useful together than separately. The tutorial supplies a route from “what is a pangenome” to a working graph; the pitfalls supply the part of that route that is normally transmitted only as tacit knowledge between people who have already been burned; and the viewer turns the output from a file that satisfies QC thresholds into something a reader can look at and interrogate.

We think two design choices are worth arguing for. The first is **using real data at a real scale**. A toy dataset will not produce an under-compressed graph when a parameter silently fails to apply, will not show a Jaccard coefficient distorted by unequal path lengths, and will not be superseded by a new assembly release in the middle of the project. Every pitfall in this report is a consequence of the data being real; none would have been teachable on a synthetic example. The cost is that a reader needs HPC access and most of a day of compute, which we accept.

The second is treating **verification against an external expectation** as part of the workflow rather than as a bonus. Because Satsuma is a known F1 of Kishu and Kunenbo, the graph can be checked against a fact that was not used to build it. Internal metrics — compression, node counts, path preservation — tell you that a graph is technically sound, but they cannot tell you that it is biologically meaningful; a pedigree can. Most pangenome projects have some equivalent external handle (a phylogeny, a known duplication, a mapping panel), and we would encourage building the QC around it.

The viewer illustrates a third point, developed in its own section above: a purpose-specific interface built in hours through interaction with AI can be the efficient choice for exploratory work with a bounded scope, without displacing the case for general-purpose, maintainable infrastructure where the data, users and lifetime are diverse. The measures it exposes — shared-node similarity and path switching — are exploratory signals that indicate where to look, not evidence of parental origin, and the viewer’s Help page says so explicitly.

Several limitations are worth stating plainly. The tutorial builds graphs with PGGB only; the Minigraph-Cactus route is described in an appendix but has not been executed on this data, so the appendix should be read as a plan rather than a result. We do not handle raw reads, which puts the tools that would properly adjudicate haplotype leakage — Merqury hap-mers, false-duplication rates — out of scope, and leaves assembly-side anomalies as flagged caveats rather than resolved questions. Graphs are built per chromosome, which is the right trade-off here but structurally cannot represent interchromosomal rearrangement. And the pedigree test establishes which parent contributed a haplotype, not which parental haplotype a given segment descends from; the windowed ancestry analysis that addresses the latter is sensitive to phase reorganisation in the parental assemblies, as the r1.0/r2.0 comparison demonstrated.

The obvious next step for the material is to carry the r1.0 and r2.0 runs side by side to the end, as a worked example of the one-thing-at-a-time discipline; the provenance table and samplesheet already make this a matter of changing six paths. Beyond that, cheaper re-runs (for example by reusing alignments with `impg` instead of repeating `wfmash`), execution of the Minigraph-Cactus appendix, and generalising the viewer's data-conversion step so that it can ingest other chromosome-level GFA sets are all within reach. Contributions, corrections and translations are welcome through the repository's issue tracker.

Acknowledgements

We thank the DBCLS BioHackathon 2026 organisers for the venue and for the opportunity to work on this project. We are grateful to Prof. Masao Nagasaki (Kyushu University) and the members of his laboratory for providing the computing environment on which the pangenome graphs in this work were built; without access to a server of that size the analysis could not have been attempted. The assemblies used throughout are those of Isobe et al. (2023), distributed by Plant GARDEN (Kazusa DNA Research Institute) and, for release r2.0, by the Mikan Genome Database 2 (MiGD2) of NARO; we are grateful to the authors and the database operators for making them openly available. Parts of the tutorial text and of the viewer were developed through dialogue with AI assistants, with all technical claims verified against the data and the tool outputs described above.

Author contributions

Shuto Machida: Conceptualization, Analysis, Writing – review & editing; Mayumi Kamada: Analysis, Writing – original draft; Yuki Moriya: Development, Writing – original draft; Toshiaki Katayama: Validation, Writing – review & editing.

References

- Cheng, H., Concepcion, G. T., Feng, X., Zhang, H., & Li, H. (2021). Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 18, 170–175 [cito:usesMethodIn].
- Garrison, E., Guarracino, A., Heumos, S., et al. (2024). Building pangenome graphs. *Nature Methods*, 21, 2008–2012. <https://doi.org/10.1038/s41592-024-02430-3> [cito:usesMethodIn].
- Garrison, E., Sirén, J., Novak, A. M., et al. (2018). Variation graph toolkit improves read mapping by representing genetic variation in the reference. *Nature Biotechnology*, 36, 875–879 [cito:usesMethodIn].
- Golicz, A. A., Bayer, P. E., Barker, G. C., et al. (2016). The pangenome of an agronomically important crop plant *Brassica oleracea*. *Nature Communications*, 7, 13390 [cito:citesAsAuthority].
- Guarracino, A., Heumos, S., Nahnsen, S., Prins, P., & Garrison, E. (2022). ODGI: understanding pangenome graphs. *Bioinformatics*, 38(13), 3319–3326 [cito:usesMethodIn].
- Hickey, G., Monlong, J., Ebler, J., et al. (2024). Pangenome graph construction from genome alignments with Minigraph-Cactus. *Nature Biotechnology*, 42, 663–673 [cito:usesMethodIn].
- Huang, Y., He, J., Xu, Y., et al. (2023). Pangenome analysis provides insight into the evolution of the orange subfamily and a key gene for citric acid accumulation in citrus fruits. *Nature Genetics*, 55, 1964–1975. <https://doi.org/10.1038/s41588-023-01516-6> [cito:citesAsRelated].
- Isobe, S., Fujii, H., Shirasawa, K., Kawahara, Y., Endo, T., & Shimada, T. (2023). Haploid-resolved and chromosome-scale genome assembly in *Citrus unshiu* and its parental species, *C. nobilis* and *C. kinokuni*. *bioRxiv*. <https://doi.org/10.1101/2023.06.02.543356> [cito:citesAsDataSource].
- Li, H., Feng, X., & Chu, C. (2020). The design and construction of reference pangenome graphs with minigraph. *Genome Biology*, 21, 265 [cito:usesMethodIn].
- Liao, W.-W., Asri, M., Ebler, J., et al. (2023). A draft human pangenome reference. *Nature*, 617, 312–324 [cito:citesAsAuthority].
- Tettelin, H., Maignani, V., Cieslewicz, M. J., et al. (2005). Genome analysis of multiple pathogenic isolates of *Streptococcus agalactiae*: implications for the microbial “pan-genome”. *Proceedings of the National Academy of Sciences*, 102(39), 13950–13955 [cito:citesAsAuthority].